Soft Heap

Proyecto EC 1.1 — Implementación, Demo y Análisis

Estructuras de Datos y Algoritmos

¿Qué es el Soft Heap?

- **Bernard Chazelle, 2000**
- Un heap de prioridad aproximado
- Permite corromper algunas claves (aumentarlas) a cambio de operaciones más rápidas
- Garantía: a lo sumo ε·n elementos corruptas en cualquier momento

$O(\log 1/\varepsilon)$

amortizado por operación

vs Binary Heap

$O(\log n)$

vs Fibonacci Heap

$O(1)$ amort.

Estructura Interna

SoftHeap

Clase principal. Lista de árboles ordenados por rank

Tree

Lista doblemente enlazada. Tiene root Node, puntero sufmin y rank

Node

Nodo de árbol binario. Contiene ckey (corrupted), rank, size, nelems, hijos left/right

Cell

Elemento de lista enlazada. Tiene key original, dato y puntero next

```
struct Cell {
    double key;
    T      data;
    Cell*  next;
};

struct Node {
    Node*  left, *right;
    Cell*  first, *last;
    double ckey;
    int    rank, size, nelems;
};

struct Tree {
    Tree*  prev, *next;
    Tree*  sufmin;
    Node*  root;
    int    rank;
};

class SoftHeap {
    Tree*  first_;
    int    top_rank_;
    double epsilon_;
    int    r_;

};
```

ε y la Corrupción

ε ∈ (0, 0.5] — ¿por qué ese rango?

ε = 0 - sin corrupción, sin ventaja
ε > 0.5 - más de la mitad corrupta, inútil

(0,0.5] → único rango con balance real

$r = \lceil \log_2(1/\varepsilon) \rceil + 5$

rank ≤ r → size = 1 - exacto

rank > r → size = ⌈3/2·size_ant⌉ - corrupto

ε = 0.5 → r = 1 + 5 = 6

rank ≤ 6 → exactos
rank > 6 → corruptibles

ε = 0.1 → r = 4 + 5 = 9

rank ≤ 9 → exactos

ε = 0.01 → r = 7 + 5 = 12

rank ≤ 12 → exactos

Mayor ε → r pequeño → más rápido
Menor ε → r grande → más exacto

Sift

```
void sift(Node* x) {
    while (x->nelems < x->size && !is_leaf(x)) {
        // paso 1-> elegir hijo con menor ckey
        if (!x->left ||
            (x->right && x->left->ckey > x->right->ckey))
             swap(x->left, x->right);


        // paso 2-> mover lista entera del hijo
        move_list(x->left, x);

        // paso 3-> actualizar ckey del padre
        x->ckey = x->left->ckey;


        // paso 4-> recursar o eliminar hijo
        if (is_leaf(x->left)) {
            free_node(x->left);
            x->left = nullptr;
        } else {
            sift(x->left);
        }
    }
```

— **OBJETIVO** —
Que cada nodo tenga al menos 'size' elementos

1. ¿Nodo tiene pocos elementos?

2. Elige hijo con menor ckey

3. Roba lista entera de una vez - carpooling, no uno por uno

4. ckey del padre sube al del hijo - key < ckey CORRUPTOS

5. Recursea en el hijo vaciado Clave: mover listas enteras es lo que da la velocidad

— COMPLEJIDAD —
profundidad recursión = O(r) = O(log 1/ε)
costo por fusión = O(1) amort.
por operación = O(log 1/ε)

Por qué r garantiza ε·n

Solo nodos con rank > r se corrompen.

En árbol binomial de rank k:
nodos de rank j = C(k, j)

Nodos con rank > r:
$\Sigma\ C(k,j)\ \ j = r+1..k$
$= \Sigma\ C(k,j)\ \ j = 0..k-r-1$ (simetría Pascal)
$= 2^{(k-r-1)}$
$\leq n / 2^r$

Necesitamos:
$n/2^r \leq \varepsilon \cdot n$
$2^r \geq 1/\varepsilon$
$r \geq \log_2(1/\varepsilon)$

Verificación numérica:

$\varepsilon=0.5$ $r=6$ $n/2^6 = n/64 \leq 0.5n$
$\varepsilon=0.1$ $r=9$ $n/2^9 = n/512 \leq 0.1n$
$\varepsilon=0.01$ $r=12$ $n/2^{12} = n/4096 \leq 0.01n$

Mayor r → más nodos exactos
→ menos corruptibles
→ garantía más estricta

Por qué 3/2 garantiza ε·n

Necesitamos que las listas no crezcan demasiado rápido para no corromper más de ε·n elementos.

Si el factor fuera 2:
size(r+1) → 2
size(r+2) → 4
size(r+3) → 8
size(r+4) → 16
crece como 2^(k-r) → demasiados corruptos

Con factor 3/2:
size(r+1) → 2
size(r+2) → 3
size(r+3) → 5
size(r+4) → 8
crece como (3/2)^(k-r) se dobla cada 2 rangos, no cada 1

QUE QUEREMOS ACOTAR:

total de items corruptos en el heap
R = { v : rank(v) > r }

|R| ≤ n/2^r -> #nodos de R

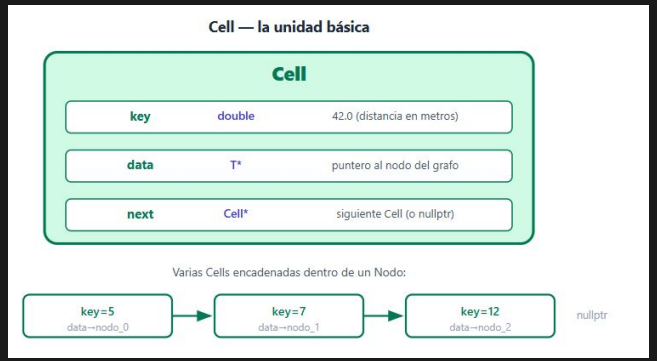
Lema 4.1
|lista(v)| ≤ 2^(⌈rank(v)/2⌉ - r/2)

Lema 4.3:
Σ 2^(rank(v)/2) ≤ 4|S|, para v ∈ S
vale para cualquier árbol binomial S

PASO 5 - juntando todo, para { v : rank(v) > r} :
Σ |lista(v)| ≤ 2 · Σ 2^((rank(v)-r-1)/2)
≤ 2 · 4|R| ← Lema 4.3
= 8|R|≤ 8 · n/2^r = n / 2^(r-3) ≤ ε·n

Implementación C++

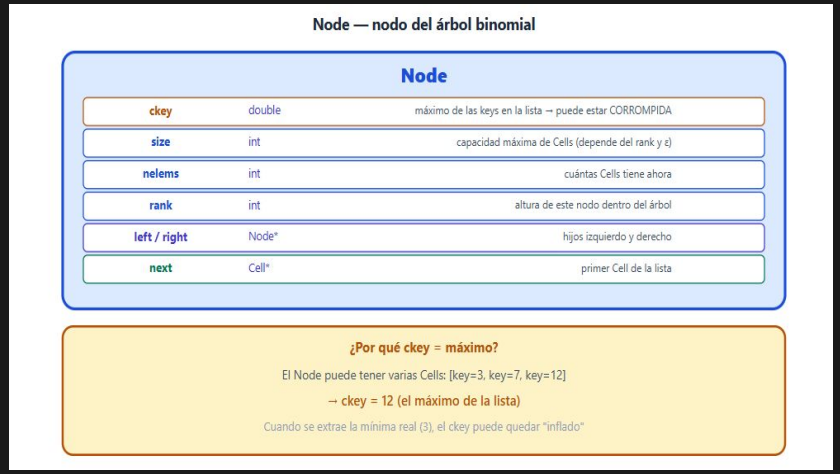
- Template genérico: SoftHeap<T>
- 3 structs anidados: Cell, Node, Tree
- SoftHeap como clase con lista de Trees

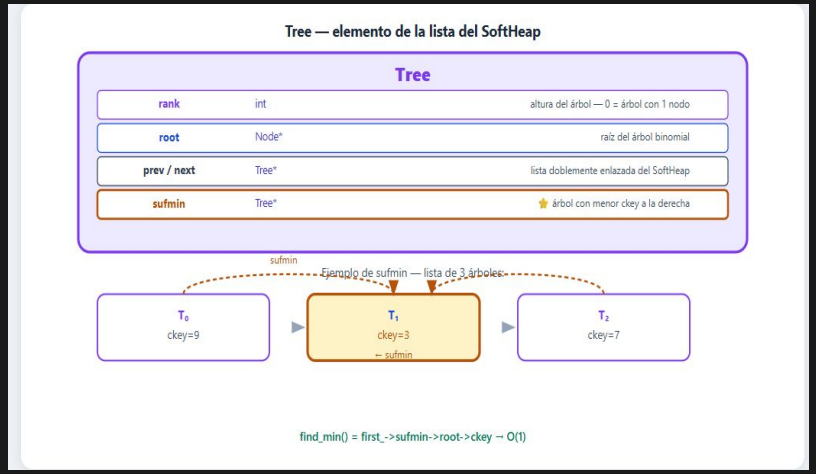


soft_heap.h — estructura

```
template <typename T>
class SoftHeap {
 struct Cell {
  double key; T data; Cell* next; };
 struct Node {
  Node *left, *right;
  Cell *first, *last;
  double ckey; int rank, size, nelems; };
 struct Tree {
  Tree *prev, *next, *sufmin;
  Node* root; int rank; };
 Tree* first_;
 int top_rank_, r_;
 double epsilon_;
public:
 explicit SoftHeap(double eps=0.5);
 void insert(double key, const T& d);
 pair<double,T> delete_min();
 void meld(SoftHeap& other);
};
```

Implementación C++

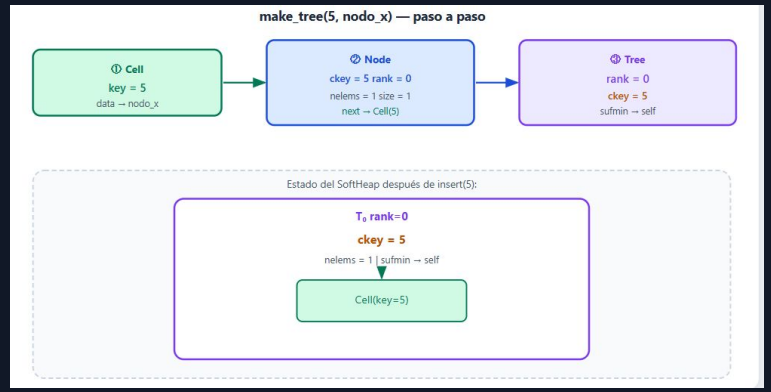




Operaciones

Operación	Descripción	Complejidad
`insert(key, data)`	Crea árbol de rank 0, llama meld	O(log 1/ε) amort.

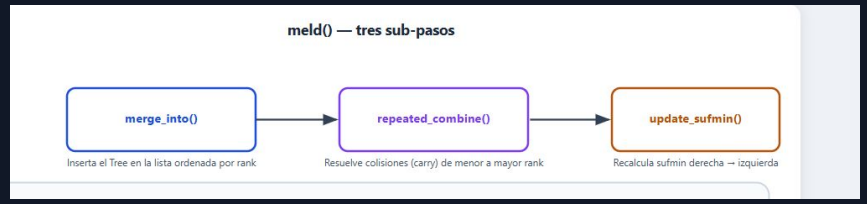
```
void insert(double key, const T& data) {
    Tree* t = make_tree(key, data);
    if (!first_) {
        first_    = t;
        top_rank_ = 0;
    } else {
        SoftHeap tmp(epsilon_);
        tmp.first_    = t;
        tmp.top_rank_ = 0;
        tmp.r_        = r_;
        meld(tmp);
    }
}
```



Operaciones

Operación	Descripción	Complejidad
`meld(other)`	Crea árbol de rank 0, llama meld	O(log n)

```
void meld(SoftHeap& other) {
    if (!other.first_) return;
    if (!first_) {
        first_     = other.first_;
        top_rank_ = other.top_rank_;
        other.first_     = nullptr;
        other.top_rank_ = -1;
        return;
    }
    int smaller_rank;
    if (top_rank_ >= other.top_rank_) {
        smaller_rank = other.top_rank_;
        merge_into(&other, this);
    } else {
        smaller_rank = top_rank_;
        merge_into(this, &other);
        first_     = other.first_;
        top_rank_ = other.top_rank_;
    }
    other.first_     = nullptr;
    other.top_rank_ = -1;
    repeated_combine(this, smaller_rank);
}
```



P: [r1] → [r3] → [r5]
Q: [r2] → [r4] → [r6]

paso 1: inserta r1 antes de r2
→ Q: [r1]→[r2]→[r4]→[r6]

paso 2: inserta r3 antes de r4
→ Q: [r1]→[r2]→[r3]→[r4]→[r6]

paso 3: inserta r5 antes de r6
→ Q: [r1]→[r2]→[r3]→[r4]→[r5]→[r6]

Operaciones

Operación	Descripción	Complejidad
`meld(other)`	Crea árbol de rank 0, llama meld	O(log n)

Complejidad:
$T(n) = O(k1+k2)+O(1) = O(k1+k2)$

¿Por qué $k \leq \log n$ en un Soft Heap?
árbol de rango r -> tiene al menos 2^r nodos
$n \geq 2^{r_1} + 2^{r_2} + \ldots + 2^{r_{\square}} \geq 1 + 2 + 4 + \ldots + 2^{(k-1)} = 2^k - 1$

$n \geq 2^k - 1$
$\Rightarrow 2^k \leq n + 1$
$\Rightarrow k \leq \log_2(n + 1)$
$\Rightarrow k = O(\log n)$

$O(k1 + k2) = O(\log n1 + \log n1) = O(\log n)$

Operaciones

Operación	Descripción	Complejidad
`delete_min()`	Crea árbol de rank 0, llama meld	O(log 1/ε) amort.

```
pair<double, T> delete_min() {
    if (!first_) throw runtime_error("heap esta vacio");

    Tree* Tmin = first_->sufmin;
    Node* x  = Tmin->root;

    auto result = pop_item(x);

    if (x->nelems <= x->size / 2) {
        if (!is_leaf(x)) {
            sift(x);
            update_sufmin(Tmin);
        } else if (x->nelems == 0) {
            free_node(x);
            Tmin->root = nullptr;
            remove_tree(Tmin);
            if (!Tmin->next)
                top_rank_ = Tmin->prev ? Tmin->prev->rank : -1;
            if (Tmin->prev) update_sufmin(Tmin->prev);
            delete Tmin;
        }
    }
    return result;
}
```

Operaciones

Operación	Descripción	Complejidad
`delete_min()`	Crea árbol de rank 0, llama meld	**O(log 1/ε) amort.**

Complejidad:
$T(n) = O(\log n) + O(k) + O(1) = O(\log n + k) = O(\log n)$

Recordar:
$r_max = \lceil \log_2(1/\varepsilon) \rceil + 5$
$k \leq r_max = O(\log 1/\varepsilon)$
$T_amort = O(\log 1/\varepsilon)$

Operaciones

Operación	Descripción	Complejidad
`find_min_key()`	Lee sufmin→root→ckey	O(1)

```
double find_min_key() {
    if (!first_) throw runtime_error("heap esta vacio");
    return first_->sufmin->root->ckey;
}
```

sufmin: puntero al árbol con menor ckey, actualizado en cada operación

Dijkstra con Soft Heap

```cpp
// Retorna dist[] y prev[] para todos los nodos
// ============================================================
struct DijkResult {
    std::vector<double> dist;  // distancia minima a cada nodo
    std::vector<int>    prev;  // nodo anterior en el camino optimo
    double              ms;
};

DijkResult dijkstra_std(
    const std::vector<std::vector<std::pair<int,double>>>& adj,
    int n, int src)
{
    auto t0 = std::chrono::high_resolution_clock::now();
    DijkResult r;
    r.dist.assign(n, INF);
    r.prev.assign(n, -1);
    std::priority_queue<
        std::pair<double,int>,
        std::vector<std::pair<double,int>>,
        std::greater<>> pq;
    r.dist[src] = 0;
    pq.push({0.0, src});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > r.dist[u]) continue;
        for (auto& [v, w] : adj[u])
            if (r.dist[u]+w < r.dist[v]) {
                r.dist[v] = r.dist[u]+w;
                r.prev[v] = u;
                pq.push({r.dist[v], v});
            }
    }
    r.ms = std::chrono::duration<double,std::milli>(
        std::chrono::high_resolution_clock::now()-t0).count();
    return r;
}
```

Prim MST con Soft Heap

```cpp
struct PrimResult {
    double           total_w;  // peso total del MST
    std::vector<int> parent;   // arbol: parent[v] = predecesor de v
    double           ms;
};

PrimResult prim_std(
    const std::vector<std::vector<std::pair<int,double>>>& adj,
    int n, int src=0)
{
    auto t0 = std::chrono::high_resolution_clock::now();
    PrimResult r; r.total_w=0;
    r.parent.assign(n,-1);
    std::vector<double> key(n,INF);
    std::vector<bool>   inMST(n,false);
    std::priority_queue<
        std::pair<double,int>,
        std::vector<std::pair<double,int>>,
        std::greater<>> pq;
    key[src]=0; pq.push({0.0,src});
    while (!pq.empty()) {
        auto [d,u] = pq.top(); pq.pop();
        if (inMST[u]) continue;
        inMST[u]=true; r.total_w+=d;
        for (auto& [v,w] : adj[u])
            if (!inMST[v] && w<key[v]) {
                key[v]=w; r.parent[v]=u; pq.push({w,v});
            }
    }
    r.ms = std::chrono::duration<double,std::milli>(
        std::chrono::high_resolution_clock::now()-t0).count();
    return r;
}
```

Prim MST con Soft Heap

Construcción del árbol de expansión mínima usando SoftHeap

Prim Exacto

priority_queue estándar

Garantiza la arista de menor peso

$O((V + E) \log V)$

MST óptimo

Prim SoftHeap ε=0.1

SoftHeap: claves corruptas pueden incluir aristas no mínimas

MST aproximado

$O(E + V \cdot \log(1/\varepsilon))$

! El MST resultante puede tener peso mayor al óptimo, pero la diferencia está acotada por ε — hay garantía teórica sobre el error introducido por la corrupción de claves.

Demo Visual — Miraflores, Lima

- **G** Grafo real extraído con OSMnx
- **N** Nodos: intersecciones de calles
- **A** Aristas: segmentos con distancia (m) y tiempo (s)
- **H** Visualización interactiva con Folium

```
Generado con Python + Folium →
miraflores_softHeap.html
```

Capa	Color	Descripción
Heatmap	**Rojo→Azul**	Distancia desde origen
Dijkstra exacto dist.	**Azul**	Ruta más corta (dist.)
Dijkstra SH ε=0.1	**Morado**	Ruta aproximada
Dijkstra exacto tiempo	**Naranja**	Ruta más rápida
MST Prim exacto	**Verde**	Árbol de expansión mínima
MST Prim SH ε=0.1	**Lima**	MST aproximado

Demo Visual — Miraflores, Lima

google.com/maps/dir/-12.103580,+-77.031558/Mal.+de+la+Marina,+Miraflores+15074/@-12.1111336,-77....

Google | YouTube | Github | Docker | Google Gemini | ChatGPT | Classroom | NotebookLM | WhatsApp | Colab | Traductor de Google | DeepSeek | PENDIENTES GRUP... | Claude

Mejor | 14 min | 31 min | 34 min | — | —

Av. Sta. Cruz 37, Lima 15073

Mal. de la Marina, Miraflores 15074

Salir ahora | Opciones

Enviar indicaciones a Xiaomi 2201117TL | Copiar enlace

13:16-13:47 — **31 min**

Regular C > 10A - 1493 - Etrascpsa

A las 13:24 desde Estación Aramburú / Comunidad Andina

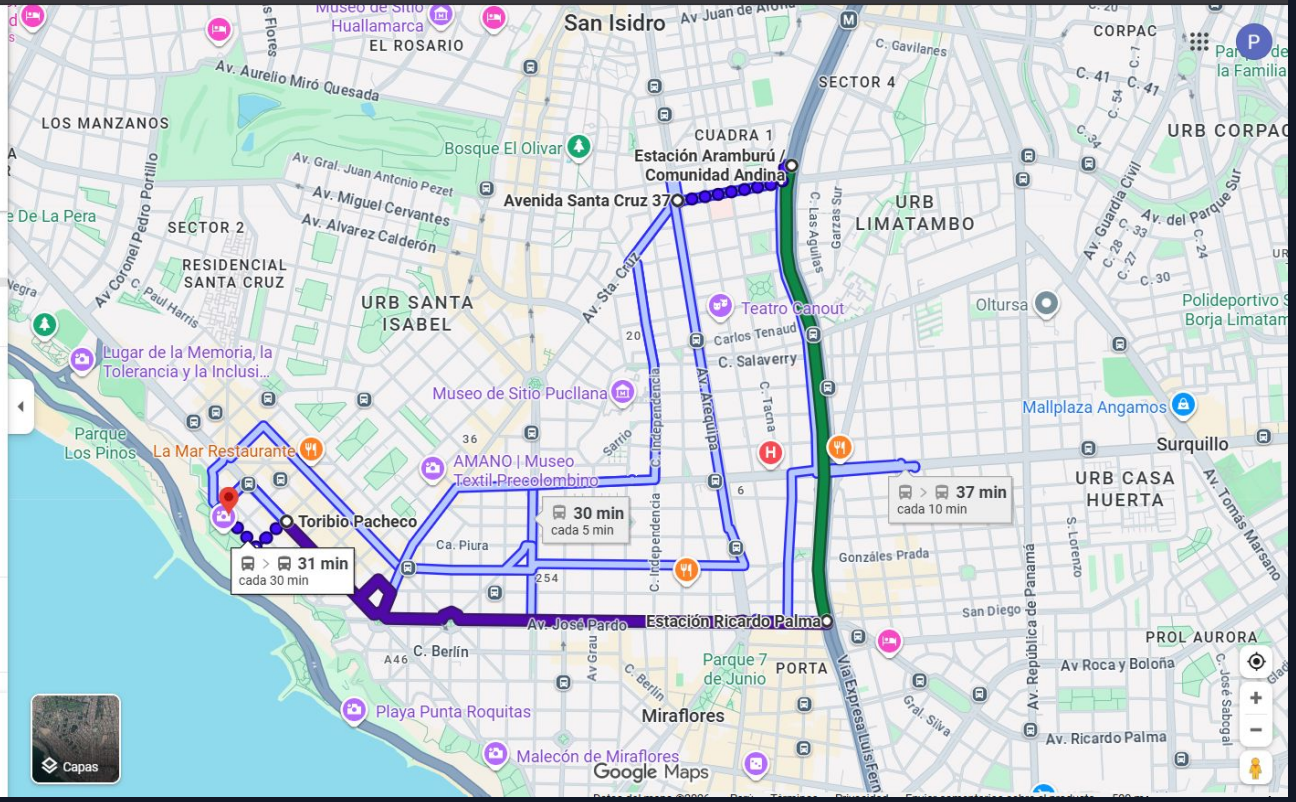
13 min — cada 30 min

Detalles

13:18-13:48 — **30 min**

1179 / 7505

¡Nuevo! Continúa tu trayecto tocando la notificación en el teléfono para obtener indicaciones



Resultados y Comparación

Dijkstra Exacto

Exacto

Tiempo (ms)

Resultado óptimo garantizado

Dijkstra SH ε=0.1

Aprox.

Tiempo (ms)

Error promedio ≈ ε del grafo

Prim Exacto

Exacto

Peso MST (m)

Menor árbol expansión posible

Prim SH ε=0.1

Aprox.

Peso MST (m)

Diferencia ≤ ε × peso óptimo

Los valores reales dependen del grafo cargado. El Soft Heap puede ser más rápido en grafos densos a cambio de una pequeña imprecisión controlada.

Conclusiones

T

Trade-off

Velocidad vs precisión controlada por ε

A mayor ε ⟶ más rápido, menos preciso

A

Aplicaciones

MST aproximado
Clustering eficiente
Algoritmos que toleran pequeños errores

I

Implementación

Header-only C++
Template genérico
Integrado en Dijkstra y Prim en Miraflores

“El Soft Heap demuestra que sacrificar exactitud de forma controlada puede llevar a mejoras asintóticas significativas”